

A Comparative Analysis of Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) for Database Partitioning

Prof. SA Shirke
Department of Computer
Engineering

Army Institute of Technology Pune
Pune, India

Samir Game
Department of Computer
Engineering

Army Institute of Technology Pune
Pune, India

Abstract— *In contemporary distributed and cloud database systems, where data is stored in a variety of servers/nodes around the globe, the efficient database partitioning is essential for a balanced workload overhead, faster query execution, and less communication overhead. In this paper, we showcase how to optimize database partitioning using two metaheuristic method: (1) Genetic Algorithm (GA), and (2) Particle Swarm Optimization (PSO). GA is inspired by the process of natural evolution and survival of the fittest. It generates a balanced and uniform partitions strategy by exploring a wide range of possible data distributions. PSO on the other hand, uses the behavior of the swarms, which helps it to find near end optimal partition of the data quicker. The performance of both the algorithms is compared by parameters such as fitness value, convergence speed, partition size. The results show us that GA gets more uniform partitions, while PSO gives us faster convergence and adaptability. Comparative analysis shows us that GA is more useful in static and large-scale database whereas PSO is better for the time-sensitive and adaptive partitioning scenarios.*

Keywords—*Genetic Algorithm, Particle Swarm Optimization, Metaheuristic Optimization, Database Partitioning, Query Optimization, Clustering, Scalability, Big Data, Performance Tuning*

Introduction

In today's world where everything is digital and everything is stored on the internet, the size and the complexity of the databases have increased at a rapid pace this happens due to massive growth of data in the fields like business, healthcare, finance, science and research, etc. With the growth of **Big-Data**, the normal traditional methods of storing partitioning, and optimizing databases are no longer sufficient to handle the needs of the modern applications [6]. Poorly optimized and non-efficient databases can lead to slower query execution, wastage of resources and higher computational costs.

This makes it necessary to explore and study advanced techniques that can handle the need for dynamic and large-scale nature of data-intensive systems [1]. Database partitioning and optimization plays an important role, this process distributes data across multiple servers and improves query execution and faster computation which ensures scalability, faster access, and reduced latency [6]. However, finding such optimal partitioning and query plan is considered a NP-hard problem, which make traditional and conventional deterministic approaches unusable of the real world large-scaled databases.

Therefore, to overcome these challenges, the researches have turned to metaheuristic algorithms which are inspired by the natural processes. Two of the most widely applied approaches are **Genetic Algorithms (GA)** and **Particle Swarm Optimization (PSO)** [3], [4]. Genetic Algorithm takes the inspiration from the Darwin's theory of natural selection, where the solution is over the course of multiple generations. The best solution is prepared by mutation, crossover and selection. This makes GA more effective in the wide search and complex scenarios [3].

On the other hand, Particle Swarm Optimization (PSO) is made on the basis of collective behaviour of bird flocks and fish schools. Where the individual particles decide their position influenced by their personal own experience and the neighbours. Compared to GA, PSO is simpler in terms of computation and often converges faster [7] and shown great promise in time bound applications.

Both GA and PSO has already been used in various areas of database optimization including **query optimization, clustering, partitioning, load balancing, and indexing** [6]. Query optimization with the help of these algorithms helps (minimize) the response time and generate the better and efficient execution plan. Data partitioning also becomes more balanced, efficient and scalable with the help of these algorithms. When GA or PSO is used, they distribute the data across several servers; however, at the same time they reduce the communication overhead [5]. For indexing and clustering tasks, both of these algorithms have been used to build structures to allow faster access to the data while being storage efficient [1], [7]. EAs are suitable when exploring complex and large solution spaces; conversely, PSOs reach a solution faster and have fewer tuning parameters [2]. This makes PSOs an attractive solution to use in real-time [8].

However, both of these techniques has limitations. GA has a problem of high computational cost which is due to evaluate large populations across many generations and thus have a slow convergence speed [9]. PSO is a quicker convergence algorithm and explore less than GA and falls into a local optima [4]. These weaknesses began to show signs of comparison for database partitioning, the algorithm we hoped to find is scalable, efficient, accurate and most importantly balanced. Therefore comparison of GA and PSO becomes very prominent for researchers or other operators in this field. The objective of this research is to provide a detailed comparison of two algorithms, Genetic Algorithm (GA) and Particle Swarm Optimization (PSO), within the domain of database optimization, particularly database partitioning. By

running each algorithm and comparing them based on convergence speed, scalability, fitness, partition variance we can identify strengths and weaknesses of the methods. Doing so adds to the academic understanding of metaheuristic algorithms for database systems and contributes to practice of heuristically selecting an optimization strategy to be used within database systems, where efficiency, accuracy and adaptability is important.

Literature Survey

The literature review presents the different situations whereby all the advanced algorithms are applied with the intentions of partitioning databases. Such algorithms are Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). These algorithms show various potentials in addressing large complex problems within a distributed, large-scale environment and security-constrained environment. This paper demonstrates how metaheuristics can be gamed in order to generate assorted and novel solutions in situations where the traditional method cannot be used. One body of work has attempted to provide theoretical assurances of evolutionary clustering.

The paper [1], solves this long-standing issue giving the first sound evidence of the approximation ability of the Global Simple Evolutionary Multiobjective Optimizer (GSEMO) on a set of classical clustering problems, such as fairness issues. It is a major advance towards the resolution of theory-practice gap in evolutionary computation.

Paper [2] describes Improved Particle Swarm Genetic Algorithm (IPSGA) in edge computing as a method to minimize the duration of task offloading in Vehicular Edge Computing systems to deep neural networks. The adaptive principles of PSO and GA can be combined to provide better DNN partitioning thus reducing processing delays considerably.

Similarly, hardware-level acceleration was introduced in Paper [3], in which acceleration of GA operations with the help of a memristive crossbar array is suggested. The technique leads to reduction in the number of clock cycles and justifies the appropriateness of processing-in-memory design on massive parallel evolutionary computation.

In the case of PSO, Paper [4] introduces a Convergence-Driven Adaptive Many-Objective PSO (CDA-MOPSO), and uses convergence-based methods to maintain exploration-exploit ratio. The overall outcome is a better algorithm that has higher convergence and distribution of solutions in many-objective optimization problems.

Besides that, Paper [7] employs a hybridization of K-Means++ clustering algorithm with PSO (Particle Swarm Optimization) to address the question of having some limitations addressed by the direct heuristics, such as a typical pitfall of the K-Means algorithm getting trapped in local optima. Genetic Programming (GP) is also utilized in the application of data that spans the problem.

Paper [5] discusses in- Popularity Partitioning of situ data in Positive-Unlabeled learning using EPGP. This strategy maximizes the quality of the resultant training data, as well as it increases the performance of a classifier by incorporating an error-based partitioning form of data into

the evolutionary algorithm during the training process of the dataset. Going that little further,

Paper [6] solves a notoriously difficult problem of the dynamic partitioning of data, trading stakes between privacy and utility with an evolutionary predictions/randomization of a part of the original data-based model. This is a field and development that is getting extremely significant in terms of the betterment and functionality of techniques that employ adaptive variations in the responses to information, to increase privacy in real-time without infringing on the highly essential use cases needed by database sensitive database systems. Besides the partitioning and optimization of data-related mining methods, interest has evidently been shown in searching and migratory development of safe and distributed evolutionary computation.

Paper [8] reports CrowdEC, a crowdsourcing system, which operates using distributed worker data, and can deliver optimization capacity of computation as well as incorporates checks and integrity of every worker input by implementing second-order checks of the data to reduce the level of noise and potentially adversarial input. This is extended in

Paper [9] which introduces a federated multiobjective EA, which is data driven and privacy-preserving by applying Diffie-Hellman-based mask techniques; hence sensitive data can be guarded when it is not to be exposed to external agents, but distributed data and optimization is possible when it is expected to do so.

Taken together these factors reveal the diversity of PSO, and GA in uses across clustering, data segregation, federated and hardware implementations as well as reflecting a trend towards hybridization, security conscious implementations and adaptive implementations giving better scaling and performance with increasing size and complexity in the real world systems. The two algorithms are both unparalleled when it comes to solving optimization and data partitioning problems. The strength of diversity used is also a unique one to GA and conversely the convergence speed of PSO is also high. The importance of these algorithms has become very significant in the current world that is now a BIG DATA. It is therefore also very crucial that we learn about them.

Materials and Methods

Parameters

The comparative analysis of these algorithms will be done by comparing these parameters and then we will get to know which algorithm is better in which scenario:

- **Fitness Value (Partition Variance):**

The fitness function is defined as the variance of partition sizes. Lower variance means that data is evenly distributed across partitions.

- **Convergence Speed:**
This means how quickly the algorithms has reach to near optimal solution. It's also means that how quickly the algorithm is executed taking less computation.
- **Final Partition Distribution:**
The number of items or data in each partition is analysed and checked. A uniform and balanced distribution means that effective partitioning is done, while unequal means inefficiency.
- **Control Parameters:**
 - GA: Population size, number of generations, and mutation rate are the main factors influencing exploration and exploitation.
 - PSO: Swarm size, number of iterations, inertia weight, and acceleration coefficients (cognitive and social) determine how particles explore and converge to optimal solutions.

Methodology

This research methodology will be to provide reasonable comparison and analyze the behavior of both the algorithms Genetic Algorithms (GA) and Particle swarm optimization (PSO) to the task of partitioning and optimization of the database. The goal is to place N data objects into K partitions with each partition as evenly balanced as possible. You can alternatively examine this issue in the following manner, suppose that there is a cloud- based database system, and it consist of suppose 1,00,000 data items and you desire to part this data items in 20 partition/servers which is located everywhere across the globe. You would not like to have one of your servers extremely overloaded and the other not. You would not want also that data that is already in one server to be found in this server. Thus, such algorithms and researched to address these issues. The chief aim is to distribute these data items uniformly, as much as possible, and without a single partition being overloaded with data and the other partition receiving considerably less data. The computation or the iteration of it should also be minimized in this entire process. The experiment is implemented in four key steps, which include problem setup, GA implementation, PSO implementation, and visualization of the results. The experimental process is carried out in four major stages: **problem setup**, **GA implementation**, **PSO implementation**, and **evaluation with visualization**.

1, Problem Setup

The first stage involves defining the problem environment and constraints. A dataset consisting of N items is considered, and the goal is to distribute these items across K partitions. Since exact equal division may not always be possible due to discrete item counts, our target is to minimize the imbalance.

- Each data item must belong to exactly one partition.
- The fitness of a candidate solution is defined as:
$$Fitness = Var(Partition\ Sizes)$$

where a **smaller variance** usually means more balanced partitions.

This setup creates a well-defined optimization problem that can be tackled using metaheuristic algorithms such as GA and PSO.

2. Genetic Algorithm (GA)

Natural evolution and survival of the fittest is an inspiration of the GA. Here, all the candidate solutions (chromosomes) are a mapping of all items to the partitions. The GA follows in a number of stages:

1. **Initialization** A random population of solutions is created. Partitions of items are coded in each solution.
2. **Fitness Evaluation:** To the solution, the number of items in each partition is determined and the variance in the number of items is evaluated. This variance is used as the fitness score..
3. **Selection:** The solution with the least variance will have a greater chance of selection to be parents of the new generation. This measure can guarantee good solutions.
4. **Crossover:** Two parent solutions are combined at a random crossover point to create a child. This allows mixing of partition structures from different parents, encouraging exploration of new solutions.
5. **Mutation:** With a small probability, one or more items are reassigned to different partitions at random. Mutation introduces diversity into the population and prevents the algorithm from getting stuck in a suboptimal solution.
6. **Iteration (Generations):** This process of selection, crossover, mutation and evaluation is repeated a number of generations. With time, the population develops and becomes better, converging slowly to a partitioning scheme with little variance. At the conclusion of the process, the GA produces the chromosome with the best performances, which is the most balanced partitioning identified.

By the end of the process, the GA outputs the best-performing chromosome, which represents the most balanced partitioning found.

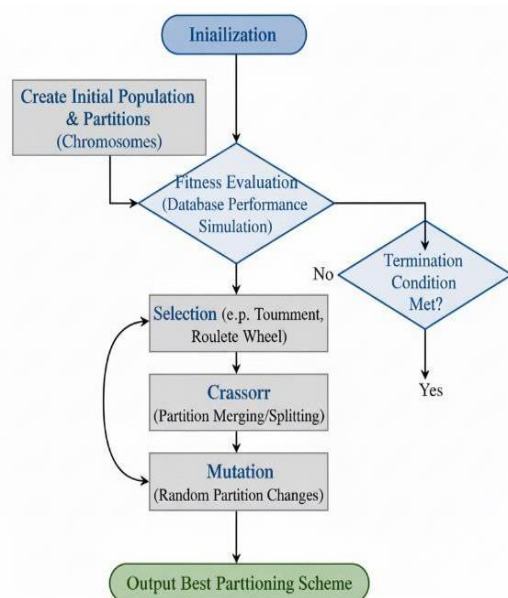


Fig 1: Workflow of GA for Database Partitioning

3. Particle Swarm Optimization (PSO)

The PSO algorithm is based on the behaviour of a flock of birds and a fish school. PSO is also used by working with a set of particles, with each particle corresponding to a single candidate partitioning of the dataset. The swarm is searching together to find an optimal solution with individual and communal knowledge.

1. Initialization: A swarm of particles is randomly generated. Each particle has a position (the current partitioning solution) and a velocity (the direction in which the solution will move).
2. Fitness Evaluation: Fitness evaluation Fitness evaluation is also similar to GA with the difference that the variance of the partition sizes is used to assess the fitness of the solution of each particle.
3. Personal Best (pBest): very particle recalls the best solution that it has achieved so far.
4. Global Best (gBest): The swarm also follows the best solution that has been discovered by one of the particles in the group.
5. Update Step: At each iteration, every particle updates its velocity and position.
 - Velocity depends on three factors: inertia (continuing in the same direction), personal experience (pBest) and social experience (gBest).
 - The new position is the new possible partitioning of the dataset.
6. Iteration (Swarm Movement): The particles change their positions in numerous iterations and they move according to their own and global knowledge to reach a balance between partitions that minimise the variance and are evenly distributed.

The final result of PSO is the global best solution, which reflects the most balanced partitioning achieved by the swarm.

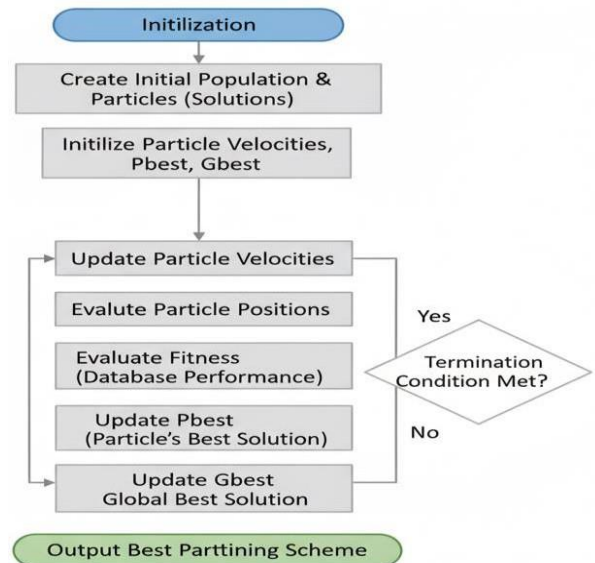


Fig 2: Workflow of PSO for Database Partitioning

4. Evaluation and Visualization

Two methods of evaluation are used to comparatively discuss GA and PSO in a meaningful manner:

Convergence Graphs: Line plots are created in order to trace the fitness value (partition variance) of the generations (in GA) and iterations (in PSO).

These graphs show the rate at which each algorithm converts to a good solution, and the stability of the solution. Final Partition Distribution: This is a display of the items in each partition at the end of the algorithms in the form of bar charts. With the help of these methods of evaluation, the search distinctions between GA and PSO are clear. GA tends to learn progressively by evolutionary operators, whereas PSO tends to learn more quickly due to the collaborative mechanism of learning.

Results and Discussion

I have compared the performance of the Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) in optimizing the partitioning of a large dataset. The output I wanted was to minimize the partition variance, which will be the fitness function for both algorithms. Below are the results from my implementation of GA and PSO, including the convergence behaviour and partitioning outcomes of the algorithms, along with graphs for the better visualisation of the results.

1. Problem Setup

I have set the problem environment as follows:

- **N = 10,000**: Total number of data items.
- **K = 20**: Number of partitions.

These two parameters are used in both GA and PSO which will tell us about the effectiveness and distribution data in K partitions and not just partitioning but also how evenly it is partitioned.

2. Convergence Analysis

- **Genetic Algorithm (GA):**

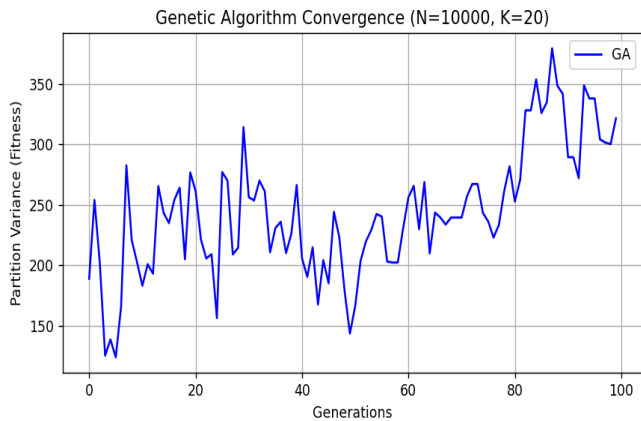


Fig 3: GA convergence plot

The Genetic Algorithm showed us a gradual increase in the partition variance by the increase in the generations. There were some fluctuations in the graph where you can see partition variance sometimes increases very much or decreases very much, this happens because the process involves randomness in mutation and crossover. These fluctuations tell GA's nature, that is explore multiple partitions and then decide final optimal solution. This was slower than PSO as GA taking more generation to achieve a final stable solution.

The GA-generated distribution graph shows that the **Genetic Algorithm achieved a nearly uniform partitioning** of data across all 20 partitions. Each partition is almost equal which tells us that all partitions are evenly distributed and uniform. There is no single partition which is overloaded or underloaded.

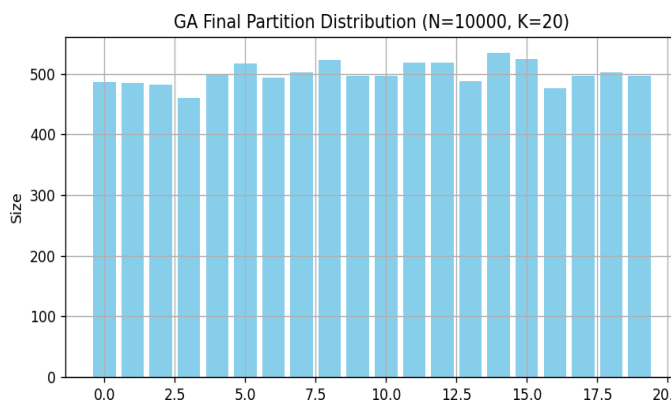


Fig 4: GA final partition distribution

- **Particle Swarm Optimization (PSO):**

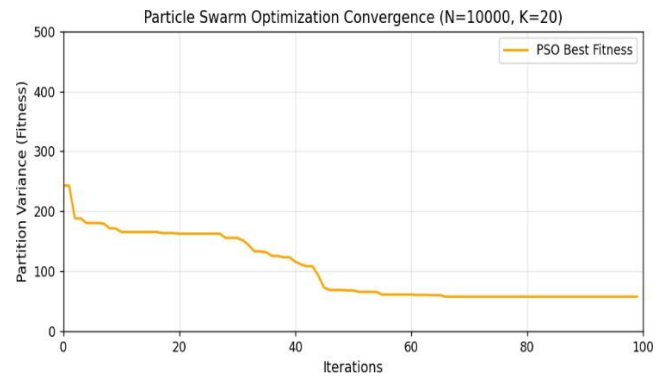


Fig 5: PSO convergence plot

PSO converged much more quickly. The convergence graph declines very rapidly and becomes almost uniform in less iterations. The fast convergence is nature of PSO, where each particle movement is influenced by its both personal and collective experience, allowing it to quickly adapt to the optimal partitioning solution.

The PSO distribution graphs shows us that distribution is **less uniform**, where some partitions have more data while others have less data. This tells us that PSO is fast in finding minimum variance but is not good in distributing the data.

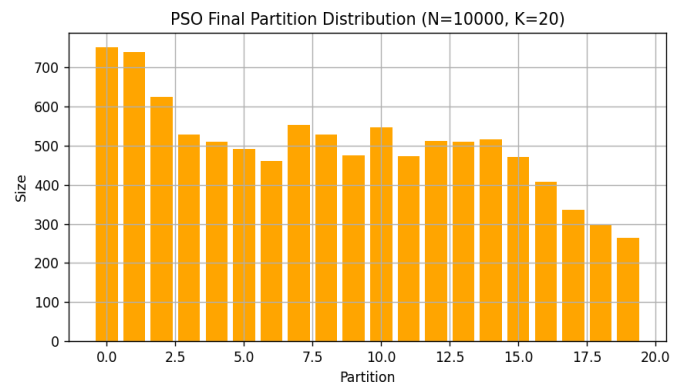


Fig 6: PSO final partition distribution

3. Discussion

a. Effectiveness of GA:

The Genetic Algorithm is a valuable algorithm for exploring the variety of different solutions. It has a very beneficial feature which helps to explore different partitioning strategies through mutation and crossover. This helps us in complex scenarios where very different types of solutions need to be evaluated. However, there is a disadvantage in this, it does not find the convergence very quickly and that is not very suitable for time bound applications. But the partition size which is evaluated by GA is very uniform and all are almost equal, there is no partition that is very loaded and also there is no partition that is not loaded at all, there is uniformity maintained.

b. Effectiveness of PSO:

PSO showed us that it has a clear advantage in terms of convergence speed. It has a ability of quickly adjust positions of particles based on both individual and global knowledge allowed it to quickly find a convergence quickly. But there is downside to it, compared to GA, PSO does not explore very much thus there can be a better solution present sometimes which can be overlooked. Data distribution done by PSO is also sometimes not uniform, some partitions can be very overloaded and some are not. So, it is fast in time bound applications.

4. Summary:

- **GA:** slower convergence, better exploration, more uniform partitions.
- **PSO:** faster convergence, less exploration, less uniform partitions.

Conclusion

In this research, one of the important challenges is tackled which is how efficiently and successfully workload or data can be balanced and distributed in a cloud-based database system. Database partitioning is important so that queries can run faster and communication overhead should be minimized. To successfully complete and overcome this challenge, we have compared two Metaheuristic optimization techniques that are used in database partitioning: Genetic Algorithm (GA) and Particle Swarm Optimization (PSO).

The results of experiments showed that GA, which is based on the natural evolution, is very effective in exploring new variety of possible partitioning strategies. It uses crossover and mutation which help to diversify the solutions. However, this comes with the downside, this algorithm does not reach optimal partitioning quickly means its convergence speed is low. But the data partitioning done by GA is uniform that is, all partition sizes are almost same, there is no partition that is overloaded with data and no partition that does not have that much data. If there is no time bound application or very large-scale application, then GA should be used.

On the other hand, PSO, which copies the social behaviour of the swarms, showed that it quicker convergence speed and can reach to the optimal partitioning quickly. But the exploration done by PSO is less than GA means there can be better partition available that PSO did not considered. PSO also sometimes does not give uniform partition sizes, that is some partitions is overloaded with data while other partitions are not. PSO can be useful for timebound applications.

To sum up, both GA and PSO are very useful Metaheuristic algorithms in database optimization and partitioning. Looking in the future, a hybrid approach can be made which give us PSO's fast convergence speed and GA's broad exploration which could help us in large scale database systems.

References

1. **Chao Qian**, "Can Evolutionary Clustering Have Theoretical Guarantees?," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 5, pp. 1220-1234, Oct. 2024. doi: 10.1109/TEVC.2023.3296645.
2. **C. Li et al.**, "DNN Partition and Offloading Strategy With Improved Particle Swarm Genetic Algorithm in VEC," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 9, pp. 5532-5542, Sep. 2024. doi: 10.1109/TIV.2023.3346506.
3. **M. Baghbanmanesh and B.-S. Kong**, "A Genetic Algorithm Accelerator Based on Memristive Crossbar Array for Massively Parallel Computation," *IEEE Access*, vol. 12, pp. 122437-122451, 2024. doi: 10.1109/ACCESS.2024.3452762.
4. **Y. Yi et al.**, "Convergence-Driven Adaptive Many-Objective Particle Swarm Optimization," *IEEE Access*, vol. 13, pp. 5129-5144, 2025. doi: 10.1109/ACCESS.2025.3525850.
5. **Y. Zhou, N. Yang, and R. Wang**, "An Embedded Unlabeled Data Partitioning PU-Learning Method Based on Genetic Programming," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 4, pp. 1858-1868, Aug. 2025. doi: 10.1109/TCSS.2024.3406377.
6. **Y.-F. Ge et al.**, "Evolutionary Dynamic Database Partitioning Optimization for Privacy and Utility," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 2296-2311, Jul./Aug. 2024. doi: 10.1109/TDSC.2023.3302284.
7. **E. Hassan et al.**, "A Hybrid K-Means++ and Particle Swarm Optimization Approach for Enhanced Document Clustering," *IEEE Access*, vol. 13, pp. 48818-48840, 2025. doi: 10.1109/ACCESS.2025.3535226.
8. **F.-F. Wei et al.**, "CrowdEC: Crowdsourcing-Based Evolutionary Computation for Distributed Optimization," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3286-3299, Nov./Dec. 2024. doi: 10.1109/TSC.2024.3433487.
9. **Q. Liu et al.**, "A Secure Federated Data-Driven Evolutionary Multi-Objective Optimization Algorithm," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 8, no. 1, pp. 191-205, Feb. 2024. doi: 10.1109/TETCI.2023.3313555.